

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from google.colab import files

uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving wdbc_data to wdbc_data

```
df = pd.read_csv("/content/wdbc.data", header=None)
```

```
print(df.head())

print(df.shape)

print(df.info())
```

```
      842302  M  17.99  10.38  122.8    1001    0.1184    0.2776    0.3001  \
0      842517  M  20.57  17.77  132.90  1326.0    0.08474    0.07864    0.0869
1      84300903  M  19.69  21.25  130.00  1203.0    0.10960    0.15990    0.1974
2      84348301  M  11.42  20.38   77.58   386.1    0.14250    0.28390    0.2414
3      84358402  M  20.29  14.34  135.10  1297.0    0.10030    0.13280    0.1980
4      843786  M  12.45  15.70   82.57   477.1    0.12780    0.17000    0.1578
```

```
      0.1471  ...  25.38  17.33   184.6    2019    0.1622    0.6656    0.7119    0.2654  \
0      0.07017  ...  24.99  23.41  158.80  1956.0    0.1238    0.1866    0.2416    0.1860
1      0.12790  ...  23.57  25.53  152.50  1709.0    0.1444    0.4245    0.4504    0.2430
2      0.10520  ...  14.91  26.50   98.87   567.7    0.2098    0.8663    0.6869    0.2575
3      0.10430  ...  22.54  16.67  152.20  1575.0    0.1374    0.2050    0.4000    0.1625
4      0.08089  ...  15.47  23.75  103.40   741.6    0.1791    0.5249    0.5355    0.1741
```

```
      0.4601    0.1189
0      0.2750    0.08902
1      0.3613    0.08758
2      0.6638    0.17300
3      0.2364    0.07678
4      0.3985    0.12440
```

```
[5 rows x 32 columns]
(568, 32)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 568 entries, 0 to 567
```

```
Data columns (total 32 columns):
```

#	Column	Non-Null Count	Dtype
0	842302	568 non-null	int64
1	M	568 non-null	object
2	17.99	568 non-null	float64
3	10.38	568 non-null	float64
4	122.8	568 non-null	float64
5	1001	568 non-null	float64
6	0.1184	568 non-null	float64
7	0.2776	568 non-null	float64
8	0.3001	568 non-null	float64
9	0.1471	568 non-null	float64
10	0.2419	568 non-null	float64
11	0.07871	568 non-null	float64
12	1.095	568 non-null	float64
13	0.9053	568 non-null	float64
14	8.589	568 non-null	float64
15	153.4	568 non-null	float64
16	0.006399	568 non-null	float64
17	0.04904	568 non-null	float64
18	0.05373	568 non-null	float64
19	0.01587	568 non-null	float64
20	0.03003	568 non-null	float64
21	0.006193	568 non-null	float64
22	25.38	568 non-null	float64

23	17.33	568 non-null	float64
24	184.6	568 non-null	float64
25	2019	568 non-null	float64
26	0.1622	568 non-null	float64
27	0.6656	568 non-null	float64
28	0.7119	568 non-null	float64

```
df[1] = df[1].map({
    'M':1,
    'B':0
})
```

```
df.drop([0], axis=1, inplace=True)
```

```
# The 'Unnamed: 32' column issue was likely due to incorrect header interpretation.
# With header=None, this specific issue should be resolved, so removing this line.
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.feature_selection import SelectKBest, f_classif

X = df.drop(columns=[1])
y = df[1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

rf_selector = RandomForestClassifier(random_state=42)
rf_selector.fit(X_train_scaled, y_train)
importances = rf_selector.feature_importances_
feature_ranking = pd.DataFrame({'Feature': X.columns, 'Importance': importances}).sort_values(by='Importance', ascending=False)

top_features = feature_ranking['Feature'].head(4).tolist()
top_indices = [X.columns.get_loc(col) for col in top_features]
X_train_selected = X_train_scaled[:, top_indices]
X_test_selected = X_test_scaled[:, top_indices]

pca = PCA(n_components=4, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
explained_variance = pca.explained_variance_ratio_

lda = LDA(n_components=1)
X_train_lda = lda.fit_transform(X_train_scaled, y_train)
X_test_lda = lda.transform(X_test_scaled)

def evaluate_model(X_tr, X_te, y_tr, y_te, name):

    lr = LogisticRegression(random_state=42)
    lr.fit(X_tr, y_tr)
    y_pred = lr.predict(X_te)

    return {
        'Approach': name,
        'Accuracy': accuracy_score(y_te, y_pred),
        'Precision': precision_score(y_te, y_pred),
        'Recall': recall_score(y_te, y_pred),
```

```

        'F1-Score': f1_score(y_te, y_pred)
    }

results = []
results.append(evaluate_model(X_train_scaled, X_test_scaled, y_train, y_test, 'All Features (Baseline)'))
results.append(evaluate_model(X_train_selected, X_test_selected, y_train, y_test, 'Selected Features (Top 4)'))
results.append(evaluate_model(X_train_pca, X_test_pca, y_train, y_test, 'PCA (4 Components)'))
results.append(evaluate_model(X_train_lda, X_test_lda, y_train, y_test, 'LDA (1 Component)'))

results_df = pd.DataFrame(results)

print("Feature Rankings:")
print(feature_ranking)
print("\nPCA Explained Variance Ratio:")
print(explained_variance, "Total:", sum(explained_variance))
print("\nModel Evaluation Results (Logistic Regression):")
print(results_df)

# Let's also check Random Forest classifier performance to see if it differs
results_rf = []
def evaluate_rf(X_tr, X_te, y_tr, y_te, name):
    rf = RandomForestClassifier(random_state=42)
    rf.fit(X_tr, y_tr)
    y_pred = rf.predict(X_te)
    return {
        'Approach': name,
        'Accuracy': accuracy_score(y_te, y_pred),
        'Precision': precision_score(y_te, y_pred),
        'Recall': recall_score(y_te, y_pred),
        'F1-Score': f1_score(y_te, y_pred)
    }

results_rf.append(evaluate_rf(X_train_scaled, X_test_scaled, y_train, y_test, 'All Features (Baseline)'))
results_rf.append(evaluate_rf(X_train_selected, X_test_selected, y_train, y_test, 'Selected Features (Top 4)'))
results_rf.append(evaluate_rf(X_train_pca, X_test_pca, y_train, y_test, 'PCA (4 Components)'))
results_rf.append(evaluate_rf(X_train_lda, X_test_lda, y_train, y_test, 'LDA (1 Component)'))

results_rf_df = pd.DataFrame(results_rf)
print("\nModel Evaluation Results (Random Forest):")
print(results_rf_df)

```

Feature Rankings:

	Feature	Importance
23	25	0.151412
27	29	0.126497
20	22	0.093475
22	24	0.083642
7	9	0.081082
2	4	0.077126
0	2	0.061990
6	8	0.050818
3	5	0.045916
26	28	0.030022
13	15	0.029064
25	27	0.018005
21	23	0.016174
1	3	0.015545
24	26	0.015093
10	12	0.014503
28	30	0.013301
5	7	0.012218
12	14	0.012037
19	21	0.006592
16	18	0.006083
15	17	0.005929
8	10	0.005097
29	31	0.004887
11	13	0.004436
4	6	0.004349
18	20	0.004242
9	11	0.003731
17	19	0.003544
14	16	0.003191

PCA Explained Variance Ratio:

[0.44593522 0.18545255 0.09584641 0.06593768] Total: 0.7931718711164907

Model Evaluation Results (Logistic Regression):

	Approach	Accuracy	Precision	Recall	F1-Score
0	All Features (Baseline)	0.964912	0.975000	0.928571	0.951220

1	Selected Features (Top 4)	0.956140	0.974359	0.904762	0.938272
2	PCA (4 Components)	0.973684	1.000000	0.928571	0.962963
3	LDA (1 Component)	0.973684	0.975610	0.952381	0.963855

Model Evaluation Results (Random Forest):

	Approach	Accuracy	Precision	Recall	F1-Score
0	All Features (Baseline)	0.973684	1.000000	0.928571	0.962963
1	Selected Features (Top 4)	0.938596	0.926829	0.904762	0.915663
2	PCA (4 Components)	0.956140	0.974359	0.904762	0.938272
3	LDA (1 Component)	0.956140	0.911111	0.976190	0.942529

Start coding or [generate](#) with AI.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

plt.clf()
sns.barplot(data=feature_ranking, x='Importance', y='Feature', hue='Feature', palette='viridis', legend=False)
plt.title('Significant Medical Parameters (Feature Importance via Random Forest)')
plt.xlabel('Importance Score')
plt.ylabel('Medical Parameter')
plt.tight_layout()
plt.savefig('feature_importance.png')

df_rf_perf = pd.DataFrame(results_rf_df)

df_melted = df_rf_perf.melt(id_vars='Approach', value_vars=['Accuracy', 'F1-Score'], var_name='Metric', value_name='Score')

plt.clf()
sns.barplot(data=df_melted, x='Approach', y='Score', hue='Metric', palette='muted')
plt.title('Comparison of Prediction Performance by Feature Technique')
plt.xlabel('Feature Engineering Approach')
plt.ylabel('Score')
plt.ylim(0, 1.0)
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig('performance_comparison.png')

print("Plots saved successfully.")
```

Plots saved successfully.



